SETUP GUIDE — Unity Space Shooter (Windows)

This guide explains exactly how to set up the scene, prefabs, UI, audio, and build pipeline for this project.

1) Create/Open Unity Project

1. Open **Unity Hub**.
2. Click **Open** and select this repository root folder.
3. Let Unity import scripts and assets.
4. Confirm project is set to **2D mode**.

2) Required Tags and Layers

Open **Edit > Project Settings > Tags and Layers** and create tags:

- `Player`
- `Enemy`
- `PlayerBullet`
- `EnemyBullet`
- `PowerUp`

(Use default layers unless you want a custom collision matrix.)

3) Physics Setup (2D)

Open **Edit > Project Settings > Physics 2D**:

- Ensure triggers are enabled by default behavior.
- Recommended: keep default gravity `(0, -9.81)` but set Rigidbody2D gravity scale to 0 on gameplay entities.

4) Sprite Asset Creation (Placeholder Art)

You can use simple placeholders:

- Player: triangle/ship silhouette
- Enemy Basic/Zigzag/Tank: 3 distinct colors/shapes
- Bullets: small circles/rectangles
- PowerUps: icon with color coding
- Green = Health
- Yellow/Orange = WeaponUpgrade
- Blue = Shield
- Explosion: radial sprite
- Background: two starfield layers for parallax

Import settings for each sprite:

- Texture Type: **Sprite (2D and UI)**
- Mesh Type: Full Rect
- Filter Mode: Bilinear (or Point if pixel art)

5) Create Core Prefabs

Create these prefabs in `Assets/Prefabs/`:

Player prefab

1. Create GameObject `PlayerShip`.
2. Add:
   - `SpriteRenderer`
   - `Rigidbody2D` (BodyType: Dynamic, Gravity Scale: 0)
   - `Collider2D` (Is Trigger: true)
   - `PlayerController`
3. Set tag to `Player`.
4. Create child `FirePoint` at ship nose.
5. Optional child `ShieldVisual` sprite (inactive by default).
6. Assign in `PlayerController`:
   - Bullet Prefab
   - Fire Point
   - Shield Visual

Bullet prefabs

Create `PlayerBullet` and `EnemyBullet` prefabs:

- Components:
- `SpriteRenderer`
- `Rigidbody2D` (Kinematic, Gravity Scale 0)
- `Collider2D` (Is Trigger true)
- `BulletController`
- Tag appropriately (`PlayerBullet` / `EnemyBullet`)

Enemy prefabs

Create 3 prefabs:

- `EnemyBasic`
- `EnemyZigzag`
- `EnemyTank`

Each has:

- `SpriteRenderer`
- `Rigidbody2D` (Dynamic/Kinematic, Gravity Scale 0)
- `Collider2D` (Is Trigger true)
- `EnemyController`
- Child `FirePoint`

Set `EnemyController.enemyType` accordingly and tune per prefab.

PowerUp prefabs

Create:

- `PowerUpHealth`
- `PowerUpWeapon`
- `PowerUpShield`

Each has:

- `SpriteRenderer`
- `Collider2D` (Is Trigger true)
- `PowerUpController`

Set `powerUpType` appropriately for each prefab.

Explosion prefab (optional)

Create `Explosion` with:

- `SpriteRenderer` (or use generated fallback)
- `ExplosionEffect`
- `AutoDestroy` (optional if you want strict lifetime enforcement)

6) Scene Setup

Create `Assets/Scenes/GameScene.unity` and add:

A) Camera

- Main Camera:
- Projection: Orthographic
- Size around 5
- Position `(0,0,-10)`

B) Managers

Create empty `GameSystems` object and add:

- `GameManager`
- `SpawnManager`
- `AudioManager`

Assign in `SpawnManager`:

- Basic Enemy Prefab
- Zigzag Enemy Prefab
- Tank Enemy Prefab

C) Environment

- Add a `ParallaxRoot` with two sprite children (layer 1, layer 2).
- Attach `ParallaxBackground` to `ParallaxRoot` and wire both renderers.
- Add `StarField` component to another empty object.

D) Gameplay Entities

- Drag `PlayerShip` prefab into scene at `(0,-3.7,0)`.

E) UI

Create a Canvas with these panels:

1. `MainMenuPanel`
   - Start button
   - Quit button
   - High score text
2. `HUDPanel`
   - Score text
   - Wave text
   - Combo text
   - Health slider + health text
   - RapidFire icon
   - Shield icon
3. `PausePanel`
   - Resume button
   - Menu button
4. `GameOverPanel`
   - Final score text
   - Final wave text
   - High score text
   - New record text
   - Restart button
   - Menu button
5. `WaveBannerPanel`
   - Wave banner text

Add `UIManager` to a UI root object and assign all references.

7) Audio Wiring

1. Keep placeholder clips in `Assets/Audio/`.
2. On `AudioManager`, assign clips:
   - Shoot
   - Explosion
   - Power-up
   - Player hit
   - Wave start
   - Game over
   - Button click

8) Validation Checklist (Play Mode)

- Player moves with WASD/Arrows.
- Spacebar fires bullets.
- Enemies spawn in waves and progress over time.

- Destroying enemies increases score.
- Fast kills increase combo multiplier.
- Power-up pickups apply effects.
- Esc toggles pause panel.
- Player death opens game-over panel.
- High score persists after restart.

9) Windows Build (.exe)

1. Open **File > Build Settings**.
2. Add `GameScene.unity` to **Scenes In Build**.
3. Platform: **PC, Mac & Linux Standalone**.
4. Target Platform: **Windows**, Architecture: **x86_64**.
5. Open **Player Settings** and set product name/icon/resolution as desired.
6. Click **Build** and select an output folder, e.g. `Builds/Windows`.
7. Run generated `Space Shooter.exe`.

10) Optional Main Menu Scene

If you want separate scenes:

- Create `MainMenu.unity` and `GameScene.unity`.
- Keep managers in game scene or use bootstrap scene.
- Update UI buttons to load scenes explicitly.

This repository is intentionally kept simple with single-scene flow for easier setup.